PROJECT REPORT

Smart Lighting System Using Arduino and LDR Sensor

Embedded Systems & IoT Internship Task
MainCrafts Technology

Prepared By	**Saurav Joshi**
Programme	B.Tech – Electronics and Communication Engineering
Institute	Bipin Tripathi Kumaon Institute of Technology, Dwarahat
Project Type	Mini Project / Simulation
Platform	Arduino UNO with LDR Sensor

Project Documentation

Table of Contents

1. Introduction

Embedded systems are an important part of modern electronic products. They combine hardware and software to perform a specific task reliably and efficiently. Unlike a general-purpose computer, an embedded system is normally designed around a particular function, such as sensing temperature, controlling a motor, managing a display, or switching a light.

The purpose of this project is to study basic embedded-system concepts and apply them in a small, practical prototype. For the mini project, an automatic lighting system is designed using an Arduino UNO, an LDR (Light Dependent Resistor), and an LED. The prototype demonstrates how a microcontroller can read a sensor and make an automatic decision based on a predefined threshold.

1.1 Objectives

• Understand the basic concept and role of embedded systems.

• Study commonly used microcontrollers and basic sensors.

• Learn how an LDR can be interfaced with an Arduino UNO.

• Develop a simple automatic lighting-control prototype.

• Understand the relationship between sensor input, program logic, and actuator output.

2. Embedded Systems Research

2.1 What is an Embedded System?

An embedded system is a dedicated computing system built into a larger product or device. It normally contains a processing unit, memory, input/output interfaces, and software designed for a particular application. The system continuously receives inputs, processes them according to its program, and produces the required output.

2.2 Real-World Applications

Area	Example Uses
Smart Home	Automatic lights, smart locks, thermostats, security sensors, and energy-management devices.
Automotive	Engine control, airbags, parking assistance, driver-assistance functions, and vehicle monitoring.
Healthcare	Wearable health trackers, patient monitoring equipment, glucose meters, and medical instruments.
Robotics	Motor control, obstacle detection, sensor processing, and autonomous movement.
Industrial Automation	Machine monitoring, process control, safety systems, and production automation.

2.3 Microcontroller vs Microprocessor

CPU, memory and peripherals are integrated on one chip.	Primarily provides the processing core; external components are commonly required.
Designed for dedicated control applications.	Commonly used for general-purpose computing.
Generally lower power and lower system cost.	Usually requires a more complex system and higher power budget.
Examples: Arduino-based boards, ESP32, Raspberry Pi Pico.	Examples: Intel Core and AMD Ryzen processor families.

3. Component Study

3.1 Microcontrollers / Development Boards

Arduino UNO

A popular beginner-friendly development board based on the ATmega328P. It provides digital and analog I/O pins and is widely used for sensor, automation, and prototype projects.

ESP32

A microcontroller platform with built-in Wi-Fi and Bluetooth. It is particularly useful when an embedded project needs wireless communication or IoT connectivity.

Raspberry Pi Pico

A low-cost microcontroller board based on the RP2040. It provides digital I/O and analog-capable interfaces suitable for embedded prototypes and sensor applications.

3.2 Basic Sensors

Temperature Sensor

Measures temperature and provides an electrical signal that can be read by a controller. It is used in weather stations, thermostats, equipment monitoring, and temperature-based automation.

Motion Sensor (PIR)

Detects movement by sensing changes in infrared radiation. It is commonly used in security alarms and automatic lighting.

Light Sensor (LDR)

An LDR changes its resistance according to the intensity of incident light. In a voltage-divider circuit, the resulting voltage can be read by an analog input and used to control lighting or other devices.

3.3 Selected Components for the Prototype

Component	Role in Project
Arduino UNO	Processes the LDR input and controls the LED.
LDR	Senses the surrounding light intensity.
10 kΩ Resistor	Works with the LDR as a voltage divider.
LED	Provides a visible indication/output.
220 Ω Resistor	Limits current through the LED.

4. Mini Project – Smart Lighting System

4.1 Project Overview

The selected mini project is an automatic lighting controller using an Arduino UNO and an LDR sensor. The LDR detects the surrounding light level and provides an analog signal to the Arduino. The Arduino compares the sensor reading with a predefined threshold and controls an LED automatically.

4.2 Project Objective

The objective is to demonstrate a simple control idea: the LDR provides an environmental input, the Arduino evaluates that input, and the LED acts as the output. A threshold value is used so that the light can be switched automatically instead of being controlled manually.

4.3 Components Required

Component	Purpose
Arduino UNO	Main microcontroller / controller board
LDR	Detects surrounding light intensity
10 kΩ resistor	Forms the LDR voltage-divider circuit
LED	Visual output / indicator
220 Ω resistor	Limits current through the LED
Breadboard	Temporary circuit assembly
Jumper wires	Electrical connections

4.4 Software / Simulation Tools

Tool	Purpose
Arduino IDE	Writing and uploading Arduino C/C++ code
Tinkercad Circuits	Simulation and circuit visualization
Embedded C/C++	Programming language used for the Arduino

5. Circuit Design and Connections

The circuit below represents the proposed simulation setup. The LDR and resistor form the sensor interface, while the Arduino UNO processes the sensor signal and controls the LED output. The following is the supplied Tinkercad-style circuit screenshot for the project documentation.

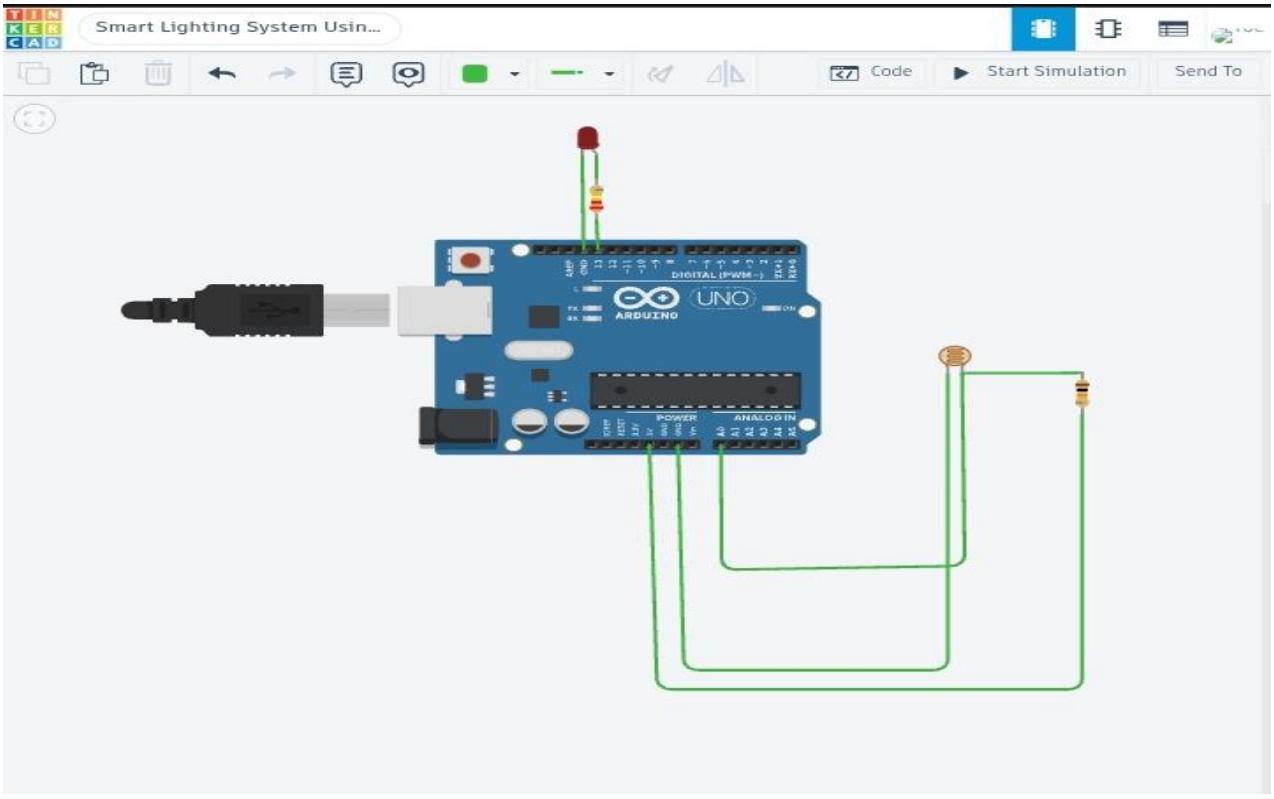

Figure 1. Tinkercad circuit simulation screenshot of the Smart Lighting System.

5.1 Connection Summary

Connection	Details
LDR	One side connected to the supply; the sensor output is connected to the Arduino analo
10 kΩ resistor	Connected with the LDR to form the voltage-divider network.
LED	Connected to a digital output through a 220 Ω current-limiting resistor.
Ground	Common ground is used for the sensor and LED circuit.
Arduino UNO	Acts as the controller and provides the required supply/reference connections.

Note: The exact analog reading depends on the LDR model and the way the voltage divider is arranged in the simulator. The program threshold is therefore adjustable.

6. Arduino Program

The following Arduino C++ program reads the LDR value from analog pin A0. A threshold of 500 is used as a starting point for the simulation. If the reading is below the threshold, the LED on pin D13 is turned ON; otherwise it is turned OFF.

```
int ldrPin = A0;
int ledPin = 13;


void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}


void loop() {
  int lightValue = analogRead(ldrPin);


  Serial.println(lightValue);
  if (lightValue > 500) {
    digitalWrite(ledPin, HIGH);
  }
  else {
    digitalWrite(ledPin, LOW);
  }
  delay(500);
}
```

7. Code Explanation

Pin declarations: ldrPin is assigned to A0 for the LDR input and ledPin is assigned to D13 for the LED output.

Threshold: The value 500 is used as an initial switching threshold. It can be adjusted after observing simulation or real sensor readings.

setup(): Sets the LED pin as an output and starts serial communication at 9600 baud so the sensor reading can be monitored.

analogRead(): Reads the LDR voltage-divider value as a number from 0 to 1023 on a standard Arduino UNO analog input.

Decision logic: When the reading is below 500, the program considers the environment dark and turns the LED ON. Otherwise, the LED is turned OFF.

delay(): A short 500 ms delay makes the output easier to observe and reduces unnecessary rapid updates.

8. Working Procedure

1. Power the Arduino UNO in the simulator or through USB when using hardware.
2. The LDR senses the surrounding light level.
3. The LDR and resistor create a voltage-divider output connected to A0.
4. Arduino reads the analog value and compares it with the threshold of 500.
5. When the value is below the threshold, the digital output is set HIGH and the LED turns ON.
6. When the value is equal to or above the threshold, the output is set LOW and the LED turns OFF.
7. The process repeats continuously so that the lighting responds automatically to changing conditions.

8.1 Working Logic

Condition	Arduino Decision	LED
Low light / LDR value < 500	Turn output HIGH	ON
Sufficient light / LDR value ≥ 500	Turn output LOW	OFF

9. Expected Result

In the simulation, the LED should automatically turn ON when the LDR reading falls below the selected threshold and turn OFF when the reading reaches or exceeds the threshold. The Serial Monitor can be used to observe the numerical sensor values and, if necessary, the threshold can be adjusted for the particular simulator or hardware setup.

10. Real-World Applications

• Automatic street lighting systems that reduce unnecessary power consumption.

• Smart home lighting that responds to ambient light conditions.

• Garden and outdoor lighting systems.

• Energy-saving lighting in corridors, parking areas, and common spaces.

• Industrial and commercial lighting control.

11. Future Improvements

• Replace the Arduino UNO with an ESP32 to add Wi-Fi/Bluetooth connectivity.

• Add a mobile or web dashboard for remote monitoring.

• Use multiple light sensors to control different lighting zones.

• Add a real-time clock or scheduling feature.

• Use a suitable relay/driver stage for controlling higher-power lamps.

• Send sensor readings to a cloud platform for data logging and energy analysis.

12. Conclusion

The Smart Lighting System demonstrates the basic workflow of an embedded system: sensing an environmental condition, processing the sensor input in a microcontroller, and controlling an output device. Using an Arduino UNO and LDR makes the idea simple enough for a beginner to understand while still representing a practical automation problem.

The project also provides practical understanding of analog input reading, threshold-based decision making, digital output control, and circuit interfacing. With an ESP32 and additional software services, the same concept can be expanded into a connected IoT lighting system.

13. References / Learning Resources

• Arduino Documentation – Getting Started and language reference.

• NPTEL – Introduction to Embedded Systems learning material.

• Tinkercad Circuits – Browser-based electronics simulation platform.

• Wokwi – Online Arduino and ESP32 simulation platform.

14. Author

Saurav Joshi

B.Tech (Electronics and Communication Engineering)

Bipin Tripathi Kumaon Institute of Technology, Dwarahat

The Project Deliverables section is intentionally omitted because the GitHub repository, LinkedIn post, and additional link are submitted separately through the MainCrafts Technology submission form.